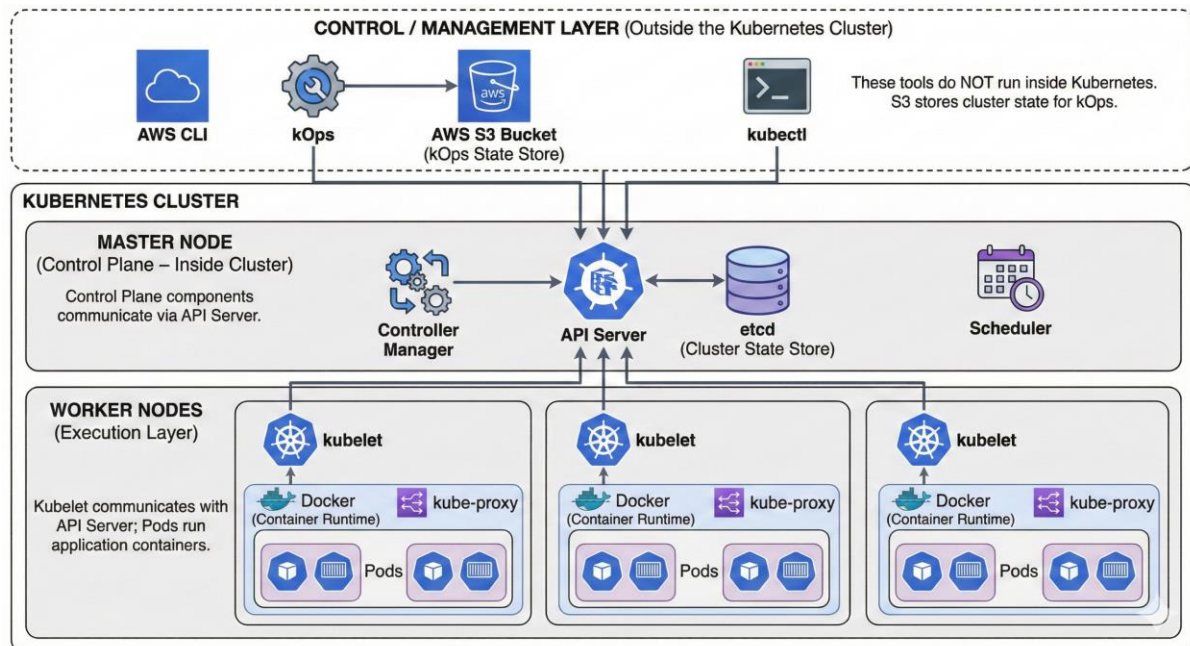


Kubernetes Architecture Explained (AWS + kOps)



Introduction

Kubernetes (K8s) is a powerful container orchestration platform used to deploy, manage, scale, and heal containerized applications. When Kubernetes is deployed on AWS using **kOps**, the architecture can be best understood by separating it into **three logical layers**:

1. Control / Management Layer (Outside the Cluster)
2. Master Node (Control Plane – Inside the Cluster)
3. Worker Nodes (Execution Layer – Inside the Cluster)

This document explains each layer in detail and also describes what happens internally when a kubectl command is executed.

Layer 1: Control / Management Layer (Outside the Cluster)

This layer consists of tools that **do not run inside Kubernetes** but are essential to create, manage, and interact with the cluster.

1. AWS CLI

The AWS Command Line Interface is used to interact with AWS services such as EC2, VPC, IAM, Load Balancers, and Route53.

Purpose:

- Provision AWS infrastructure
- Manage networking and security

Important: AWS CLI manages AWS resources, not Kubernetes directly.

2. kOps (Kubernetes Operations)

kOps is a production-grade tool used to create, upgrade, and maintain Kubernetes clusters on AWS.

Responsibilities:

- Create Master and Worker nodes
- Configure etcd
- Set up Auto Scaling Groups
- Configure IAM roles and networking

kOps acts as the **cluster lifecycle manager**.

3. AWS S3 Bucket (kOps State Store)

kOps stores the entire cluster configuration and state in an S3 bucket.

Stored Data Includes:

- Cluster configuration
- Node instance configuration
- Certificates and keys

If this S3 bucket is lost, kOps cannot manage or modify the cluster.

4. kubectl

kubectl is the command-line tool used to interact with a running Kubernetes cluster.

Functions:

- Deploy applications
- Inspect cluster resources
- Scale and manage workloads

kubectl communicates directly with the Kubernetes API Server.

Layer 2: Master Node (Control Plane – Inside the Cluster)

The Master Node hosts the **core Kubernetes control plane components**. It acts as the brain of the cluster.

1. API Server

The API Server is the entry point for all Kubernetes operations.

Responsibilities:

- Authenticate and authorize requests
- Validate YAML manifests
- Store cluster state in etcd

All kubectl commands go through the API Server.

2. etcd

etcd is a distributed key-value datastore used by Kubernetes.

Stores:

- Desired and current state of the cluster
- Pod definitions
- Secrets and ConfigMaps

Loss of etcd data means loss of the cluster state.

3. Controller Manager

The Controller Manager runs multiple controllers that continuously monitor the cluster.

Key Controllers:

- Node Controller
- Deployment Controller
- ReplicaSet Controller

Purpose:

Ensure that the actual state of the cluster matches the desired state.

4. Scheduler

The Scheduler assigns newly created pods to suitable worker nodes.

Scheduling Decisions Based On:

- CPU and memory availability
- Node affinity rules
- Taints and tolerations

The scheduler does not run pods; it only decides placement.

Layer 3: Worker Nodes (Execution Layer)

Worker nodes are where application workloads actually run.

1. kubelet

kubelet is an agent that runs on every worker node.

Responsibilities:

- Communicate with API Server
 - Ensure containers are running as per pod specifications
-

2. Docker (Container Runtime)

Docker (or containerd / CRI-O) is responsible for running containers inside pods.

kubelet instructs the container runtime to pull images and start containers.

3. kube-proxy

kube-proxy manages networking on worker nodes.

Functions:

- Service discovery
- Load balancing
- Pod-to-pod communication

Uses iptables or IPVS internally.

Conclusion

In an AWS-based Kubernetes setup using kOps, management tools like AWS CLI, kOps, S3, and kubectl operate outside the cluster. Inside the cluster, the Master Node manages scheduling, state, and control logic, while Worker Nodes execute application workloads. Understanding this layered architecture provides clarity, avoids confusion, and forms a strong foundation for Kubernetes administration and DevOps interviews.